

- DROPKAST — Credentials Checklist
 - 1. Anthropic (Claude) — Highest impact, but optional
 - 1b. Free chat providers (NVIDIA / Groq / Cerebras / OpenRouter)
 - 2. Supabase — Auth + database
 - First time: run the migrations
 - 3. Cloudinary — Audio + artwork uploads
 - 4. Stripe — Splits payouts (Phase 3)
 - 5. RouteNote — Real DSP delivery (Phase 1)
 - 6. Sentry — Error tracking (Phase 6)
 - What works WITHOUT any keys

DROPKAST — Credentials Checklist

The app **runs without any keys** thanks to graceful fallbacks (in-memory DB, mock auth, fake AI responses), but each key you add unlocks a real feature. Add them in order of impact.

For Vercel: **Project Settings** → **Environment Variables**. For local dev: copy `.env.example` to `.env`.

1. Anthropic (Claude) — Highest impact, but optional

You can run the chat assistant with **free providers** (Section 1b) instead of Claude. Claude is the primary because it supports tool use (the assistant can pull live data) and prompt caching. If you skip it, the assistant works but can only answer general questions, not pull your specific releases/analytics/campaigns.

Variable	Where	Notes
<code>ANTHROPIC_API_KEY</code>	https://console.anthropic.com/settings/keys	Server-only. Put it in Vercel and your local <code>.env</code> .

Cost expectation with prompt caching enabled (already wired):

- Sonnet 4.6 chat: ~\$0.02–0.05 per 10-message session
- Haiku 4.5 viral ideas: ~\$0.001 per call
- Sonnet A&R critique: ~\$0.05 per submission

1b. Free chat providers (NVIDIA / Groq / Cerebras / OpenRouter)

OpenAI-compatible endpoints — wired through a single adapter. Add **any** of these and the user can pick them in the assistant's provider dropdown. Useful for free fallback when Claude budget is hit.

Variable	Where	Free tier
<code>NVIDIA_API_KEY</code>	https://build.nvidia.com	40 RPM, indefinite. Llama 3.3 70B + Mistral + Qwen.
<code>GROQ_API_KEY</code>	https://console.groq.com/keys	1M tokens/day, fastest inference.
<code>CEREBRAS_API_KEY</code>	https://cloud.cerebras.ai	1M tokens/day, 8K context cap.
<code>OPENROUTER_API_KEY</code>	https://openrouter.ai/keys	50 req/day free, 1000/day with \$10 balance. Many models.

These providers do NOT support tool use through the adapter, so the assistant becomes a plain chatbot when one of them is selected. Use Claude for catalog-aware answers; free providers for general questions or when you want to save Anthropic credits.

See [/ai-providers](#) inside the app for the full tier list, pricing, and signup links.

2. Supabase — Auth + database

Replaces the localStorage mock auth and the in-memory `db` that resets on every Vercel cold start.

1. Create a project at <https://supabase.com/dashboard>
2. Wait for the database to provision (~1 minute)
3. Go to **Project Settings** → **API**

Variable	Where in Supabase	Exposed to browser?
<code>VITE_SUPABASE_URL</code>	Project Settings → API → Project URL	Yes (safe)
<code>VITE_SUPABASE_ANON_KEY</code>	Project Settings → API → <code>anon public</code> key	Yes (safe)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Project Settings → API → <code>service_role</code> key	No — server only
<code>DATABASE_URL</code>	Project Settings → Database → Connection String → URI (use the Pooled connection for serverless)	No

After adding `VITE_SUPABASE_*`, real signup/login replaces the mock. After adding `DATABASE_URL`, Drizzle takes over from in-memory storage.

First time: run the migrations

```
npx drizzle-kit generate
npx drizzle-kit migrate
```

This creates all the tables defined in `db/schema.ts`.

3. Cloudinary — Audio + artwork uploads

Optional. Without it, uploaded files use in-memory storage that vanishes when the function cools down.

Variable	Where
<code>CLOUDINARY_CLOUD_NAME</code>	https://cloudinary.com/console → Dashboard
<code>CLOUDINARY_API_KEY</code>	Same dashboard
<code>CLOUDINARY_API_SECRET</code>	Same dashboard (server-only)

4. Stripe — Splits payouts (Phase 3)

Not yet wired into the UI; add when you start Phase 3 of `IMPLEMENTATION_PLAN.md`.

Variable	Where
<code>STRIPE_SECRET_KEY</code>	https://dashboard.stripe.com/apikeys
<code>STRIPE_WEBHOOK_SECRET</code>	https://dashboard.stripe.com/webhooks → Add endpoint pointing at <code>/api/stripe/webhook</code>

5. RouteNote — Real DSP delivery (Phase 1)

Apply for the partner program — takes a few business days for approval.

Variable	Where
<code>ROUTENOTE_API_KEY</code>	https://routenote.com/distribution-partners (B2B onboarding)

6. Sentry — Error tracking (Phase 6)

Free tier covers most early-stage projects.

Variable	Where
<code>SENTRY_DSN</code>	https://sentry.io → Project Settings → Client Keys (DSN) — server projects

Variable	Where
VITE_SENTRY_DSN	Same — but for the React/browser project

What works WITHOUT any keys

- Full UI navigation
- Welcome flow + portal selection
- Mock auth (localStorage)
- All API endpoints (in-memory data, lost on cold start)
- Fake AI responses (placeholder text returned)

So you can demo the whole flow before adding a single key. Add keys when you need the real thing.